Overview

LLMOps extends MLOps for large language model systems with additional complexity:

- non-deterministic outputs,
- prompt and retrieval dependencies,
- token-based cost dynamics,
- safety/governance requirements specific to generative behavior.

A practical definition:

LLMOps is the set of practices that makes LLM-powered systems reliable, cost-aware, observable, and governable from design to production operations.

Why LLMOps is Distinct from Classical MLOps

Shared foundations

LLMOps inherits core MLOps practices:

- versioning,
- CI/CD principles,
- monitoring and incident response,
- experiment tracking.

Additional LLMOps dimensions

1. **Prompt layer**: prompts/templates are first-class artifacts.
2. **Context layer**: retrieval and chunking quality heavily influence outputs.
3. **Evaluation ambiguity**: many tasks have no single correct answer.
4. **Cost layer**: token usage and latency trade-offs are central product constraints.
5. **Safety layer**: hallucination, harmful outputs, and prompt injection risks.

LLM Lifecycle

Stage 1: Use-case and architecture selection

Decide:

- closed model API vs open model hosting,
- single-step prompting vs multi-step orchestration,
- pure generation vs RAG-augmented generation.

Stage 2: Data and context strategy

For RAG workflows, define:

- document ingestion pipeline,
- chunking and metadata policy,
- embedding model and indexing strategy,
- refresh cadence.

Stage 3: Prompt/workflow engineering

Treat prompts as versioned assets with test suites:

- system instructions,

- output schema constraints,
- tool-call policies,
- fallback behavior.

Stage 4: Evaluation and validation

Use mixed evaluation:

- automatic metrics (format validity, groundedness proxies),
- rubric-based LLM-as-judge where suitable,
- human review for high-risk tasks.

Stage 5: Deployment and scaling

Production concerns:

- concurrency and rate limits,
- timeout and retry policies,
- model routing and caching,
- rollback to prior prompt/model chain.

Stage 6: Continuous improvement

- prompt and retrieval iteration,
- guardrail tuning,
- dataset and benchmark refresh,
- cost-performance optimization.

Core LLMOps Components

Prompt and configuration registry

Track:

- prompt versions,
- model identifiers,
- temperature/top_p and output constraints,
- linked evaluation baselines.

Retrieval operations (for RAG)

Critical quality levers:

- chunk granularity,
- embedding quality,
- reranking strategy,
- stale index detection.

Inference gateway layer

A gateway can centralize:

- auth and policy checks,
- provider abstraction,
- throttling and caching,
- logging and trace IDs.

Evaluation harness

Should support:

- offline regression suites,
- scenario-based quality thresholds,
- continuous post-deploy sampling.

Cost, Latency, and Quality Trade-offs

Token economics

Approximate cost relationship:

$$\text{Inference Cost} \propto \text{Input Tokens} + \text{Output Tokens}$$

Key optimization levers:

- prompt compaction,
- retrieval precision (reduce irrelevant context),
- response-length control,
- model routing by task complexity.

Latency vs quality

- Larger models can improve quality but increase latency/cost.
- Multi-step chains may improve correctness but can violate UX SLAs.

LLMOps requires explicit SLO budgets for both latency and quality.

Evaluation Strategy for LLM Systems

Multi-layer evaluation

1. **Component-level**: retrieval recall/precision proxies, prompt format validity.
2. **End-to-end**: task success and user acceptance.
3. **Safety-level**: toxicity, policy violations, hallucination rate.

Golden datasets and regression suites

Maintain versioned, domain-specific benchmark sets:

- typical queries,
- edge-case queries,
- adversarial/prompt-injection samples.

Human + automated hybrid

Automated checks scale quickly; human audits validate nuanced quality and risk.

Production Case Studies & Exceptions

Case 1: Customer-support RAG assistant

Issue:

- good fluency but occasional unsupported answers.

Fix:

- citation-required response policy,
- abstain when retrieval confidence is low,
- stricter context filtering.

Case 2: Internal coding assistant

Issue:

- costs spiked due to verbose prompts and long context windows.

Fix:

- prompt compaction, selective retrieval, caching frequent requests.

Case 3 (Exception): Deterministic workflow preferred

Scenario:

- policy-compliance checks needed strict reproducibility.

Decision:

- use rule-based engine for final decision; LLM only for summarization support.

Interview Questions & Answers

1. **What is LLMOps?**
   Operational discipline for building, deploying, monitoring, and governing LLM systems at scale.
2. **How is LLMOps different from MLOps?**
   LLMOps adds prompt/context management, token economics, and generative risk controls.
3. **Why treat prompts as versioned artifacts?**
   Prompt changes can alter behavior as much as model changes.
4. **What are key stages of LLM lifecycle?**
   Use-case framing, data/context setup, prompt workflow design, evaluation, deployment, and continuous improvement.
5. **How do you evaluate an LLM app?**
   Use mixed automated + human evaluation across quality, safety, and business outcomes.
6. **When do you choose RAG over fine-tuning?**
   When knowledge changes frequently and citation/freshness are priorities.
7. **How do you control LLM costs?**
   Prompt optimization, retrieval precision, model routing, and caching.
8. **What is prompt injection risk?**
   Malicious input attempts to override system instructions or exfiltrate sensitive data.
9. **What is a golden dataset?**
   A curated benchmark set used for consistent regression testing.
10. **How do you improve groundedness?**
    Better retrieval, citation requirements, and abstention logic for low-confidence contexts.
11. **What is model routing?**
    Selecting different models per request complexity to balance quality and cost.

12. **Why is LLM monitoring harder than classic ML monitoring?**
    Outputs are free-form, subjective, and non-deterministic.
13. **How do you define LLM SLOs?**
    Combine latency/cost targets with quality and safety thresholds.
14. **What are common LLMOps failure modes?**
    Stale retrieval index, runaway token cost, silent quality regressions.
15. **How do you ship LLM systems safely?**
    Staged rollout, policy guardrails, continuous evals, and rollback-ready architecture.

References

- Coursera Duke LLMOps specialization: https://www.coursera.org/specializations/large-language-model-operations
- Coursera Open Source LLMOps Solutions: https://www.coursera.org/learn/open-source-llmops-solutions
- Coursera Foundations of Local LLMs: https://www.coursera.org/learn/local-large-language-models
- IBM LLMOps overview: https://www.ibm.com/think/topics/llmops
- DeepLearning.AI LLMOps course page: https://www.deeplearning.ai/courses/llmops
- Google Cloud MLOps architecture (for lifecycle parallels): https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning